

More Shell Programming Secrets Nobody Talks About



This is the second part of the article on shell programming secrets. (The first part was carried in the October 2022 issue of *Open Source For You*.)

It covers some other important behaviour of bash, particularly that of text expansions and substitutions.



The famed text-processing abilities of the Bourne shell (sh) have been vastly expanded in the Bourne-Again shell (bash). These abilities are efficient to a fault and the code can be even more cryptic. This makes the bash command interpreter and programming language very powerful but also a minefield of errors.

In the last article published in the October 2022 issue of *Open Source For You* titled ‘Shell Programming Secrets Nobody Talks About’, you learnt that:

- sh and bash are not the same
- *if* statements check for exit codes, not Boolean values
- *[*, *true* and *false* are programs, not keywords
- Presence/absence of space in variable comparisons and assignments makes quite a difference
- *[[* is not the fail-safe version of *[*
- Arithmetic operations are not straightforward as in other languages
- Array operations use cryptic operators
- By default, bash will not stop for errors

I forgot to mention that if you use the *-e* option of the *set* command, as in *set -e*, then you will not be able to check for error codes of previous statements with the *if [\$? -eq 0]; then* construct. This option makes bash stop running a shell script if any of its statements encounter an error. If you are doing error handling using *if* statements, then begin your scripts like this:

```
#!/bin/bash
set -u
# The rest of your shell script
```

The *-u* option of the *set* command, if you remember from the last article, ensures that undefined variables are not replaced with an empty string and instead cause an error.

In this article, we will focus on how bash performs text expansions and substitutions. I will only cover what